Day 16 — GitHub Actions Fundamentals

Day 16 — GitHub Actions Fundamentals Lab

Objective

Build and run a working GitHub Actions workflow from scratch, and understand

how a push event turns into a running pipeline.

Mental Model

```
Developer
  -> git push
  -> GitHub
  -> GitHub Actions
  -> Workflow
  -> Job
  -> Runner
  -> Steps
```

Tasks

1. **Create the workflow directory**

```
mkdir -p .github/workflows
```

2. **Add a basic workflow**

Copy `../../workflows/basic-ci.yml` into `.github/workflows/basic-ci.yml`.

3. **Trigger it**

Push a commit to `main` and open the **Actions** tab on GitHub to watch

the workflow run.

4. **Inspect the run**

- Identify the workflow, job, runner, and steps in the UI.
- Note the `runs-on` value and confirm which OS the runner used.
- Expand a step log and find `GITHUB_WORKSPACE`, `GITHUB_REF_NAME`.

5. **Break it on purpose**

Change `run: echo "Build step goes here"` to an invalid command (e.g.

`run: exit 1`) and push again. Observe the red X and read the failure log.

6. **Compare to Jenkins**

Write 3-4 lines comparing this run to a Jenkins pipeline run you've done

before (see `../../../07-cicd/jenkins/` notes if available): where is the server, who manages the agent/runner, how are credentials handled.

Success Criteria

- [ ] Workflow runs automatically on push
- [ ] You can identify workflow / job / runner / step in the Actions UI
- [ ] You've seen both a passing and a failing run
- [ ] You can explain triggers, jobs, steps, and runners without notes

Day 16 Interview Questions — GitHub Actions Fundamentals

Q1. What is GitHub Actions?

A CI/CD and automation platform built into GitHub that runs workflows in response to repository events (push, pull request, schedule, manual, etc.).

Q2. What is a workflow?

A YAML file in `.github/workflows/` that defines triggers, jobs, and steps — the top-level automation definition.

Q3. What is a job?

A set of steps that execute on the same runner. Jobs run in parallel by default unless linked with `needs:`.

Q4. What is a step?

A single task inside a job — either `uses:` (a reusable Action) or `run:` (a shell command).

Q5. What is a runner?

The machine (VM or container) that actually executes a job's steps. Can be GitHub-hosted (ephemeral, managed) or self-hosted (your own infrastructure).

Q6. What does `runs-on` do?

Specifies which runner a job executes on, e.g. `runs-on: ubuntu-latest`.

Q7. Difference between `uses` and `run`?

`uses` invokes a packaged, reusable Action (e.g. `actions/checkout@v4`). `run` executes raw shell commands directly on the runner.

Q8. Where must workflow files live?

`.github/workflows/*.yml` at the repository root — GitHub only discovers workflows in this exact path.

Q9. What triggers a workflow?

Events defined under `on:` — e.g. `push`, `pull_request`, `schedule`, `workflow_dispatch` (manual trigger).

Q10. Jenkins vs GitHub Actions — name two differences.

- Jenkins requires you to install, host, and maintain the server and agents; GitHub Actions runners are managed for you (when hosted).
- Jenkins pipelines are typically Groovy (`Jenkinsfile`); GitHub Actions

workflows are YAML and live natively alongside the code they build.